Project Report

Theme: Medical Prescription OCR with TrOCR

This project tackles the long-standing problem of digitizing handwritten medical prescriptions by fine-tuning a Transformer-based vision encoder–decoder (TrOCR) and exposing it through a production-ready API and web interface.

Group Members:

Benhammadi Lokmane

Khedim Youcef

1 May 2026

Contents

1 Introduction and Problem Statement **5**

1.1 The Challenge: Reading Handwritten Medical Prescriptions 5

1.2 Why Prescription OCR is Hard 5

1.3 Our Approach: Fine-Tuned Vision Encoder–Decoder 5

2 Background: OCR and Vision Encoders **7**

2.1 From Classical OCR to Deep Learning 7

2.2 The Vision Transformer (ViT) 7

2.3 Encoder–Decoder Architectures for Image-to-Text 7

3 TrOCR: Model Architecture **8**

3.1 Overview of TrOCR 8

3.2 The Image Encoder 8

3.3 The Text Decoder 9

3.4 Cross-Attention and Generation 9

3.5 Why TrOCR for Handwritten Prescriptions 9

4 Dataset **10**

4.1 Source 10

4.2 Dataset Statistics 11

4.3 Train / Validation / Test Split 12

5 Preprocessing and Line Segmentation **13**

5.1 Image Cleanup 13

5.2 Line / Region Detection 13

5.3 TrOCR Processor 13

6 Training Methodology **14**

6.1 Configuration 14

6.2 Hardware: Kaggle 2×T4 Setup 14

6.3 Loss, Optimizer, and Generation 15

6.4 Checkpointing and HuggingFace Hub Persistence 15

7 Inference Pipeline **16**

7.1 End-to-End Flow 16

7.2 Post-Processing 16

8 Evaluation and Results **17**

8.1 Metrics 17

8.2 Qualitative Examples 17

8.3 Failure Analysis 18

9 API and Frontend **19**

9.1 FastAPI Backend 19

9.2 React + Vite Frontend 20

10 Discussion and Limitations **21**

11 Technologies Used **22**

11.1 PyTorch 22

11.2 HuggingFace Transformers 22

11.3 FastAPI 22

11.4 React + Vite 22

11.5 Docker 23

11.6 Kaggle 23

11.7 GitHub 23

12 Conclusion **24**

List of Figures

1 End-to-end medical prescription OCR pipeline. 6

2 Vision Transformer (ViT): an image is split into patches and processed as a sequence. 7

3 TrOCR: a pretrained image Transformer encoder feeds a pretrained text Transformer decoder. 8

4 Examples from the prescription dataset: cropped lines with ground-truth labels. 10

5 Distribution of character lengths (left) and token lengths (right) across the prescription dataset. The 95th percentile is highlighted in both plots to inform sequence length limits. 11

6 Frequency of the top-20 most commonly prescribed medications (left) and the distribution of the number of medications per prescription (right). 11

7 Age distribution of patients represented in the dataset ($n = 1000$). 12

8 Effect of the preprocessing stage on a noisy phone-captured prescription. 13

9 Detected line regions overlaid on the input prescription. 13

10 Training metrics tracked over 5 epochs, including per-epoch training loss (top left) and granular per-batch training loss (bottom right), demonstrating stable convergence. 15

11 Baseline TrOCR (zero-shot) predictions. The model struggles with the domain-specific vocabulary and fails to output the targeted metadata structures, often resulting in severe hallucinations. 17

12 Validation predictions after fine-tuning. The model successfully transcribes cursive handwriting and correctly formats the output into the expected structural schema. 18

13 FastAPI auto-generated Swagger UI exposing `/health` and `/predict`. 19

14 Prediction results view showing extracted lines and processing time. 20

15 Sketch of a future end-to-end layout-aware OCR pipeline with drug-database normalization. . 21

1. Introduction and Problem Statement

1.1. The Challenge: Reading Handwritten Medical Prescriptions

Handwritten medical prescriptions remain one of the most stubborn last-mile problems in healthcare digitization. Despite the rise of electronic health records, a large share of prescriptions issued worldwide are still written by hand, often hastily, on small slips of paper. Misreadings at the pharmacy directly translate into dispensing errors, drug interactions, and patient harm.

Two challenges sit at the core of this project:

- **Recognition:** Doctors' handwriting is famously irregular — abbreviations, overlapping strokes, and inconsistent spacing make it hostile to off-the-shelf OCR.
- **Structure:** Even after raw text is extracted, downstream systems need structured fields (drug name, dosage, frequency) rather than a free-text blob.

The central question is: *"Can a fine-tuned vision–language model transcribe handwritten prescription lines accurately enough to be useful in a real digitization pipeline?"*

1.2. Why Prescription OCR is Hard

Three properties make prescriptions harder than typical handwritten text:

1. **Domain-specific vocabulary** — drug names rarely occur in general handwriting corpora.
2. **Layout variability** — header letterhead, table-like dosage grids, and free-form notes mix in a single image.
3. **Acquisition noise** — prescriptions are typically captured with phone cameras, introducing skew, shadows, and uneven lighting.

1.3. Our Approach: Fine-Tuned Vision Encoder–Decoder

We fine-tune Microsoft's `trocr-base-handwritten` on a public prescription dataset, wrap it in an end-to-end pipeline (preprocessing → line segmentation → TrOCR → post-processing), and expose it through a FastAPI backend and a React+Vite frontend.

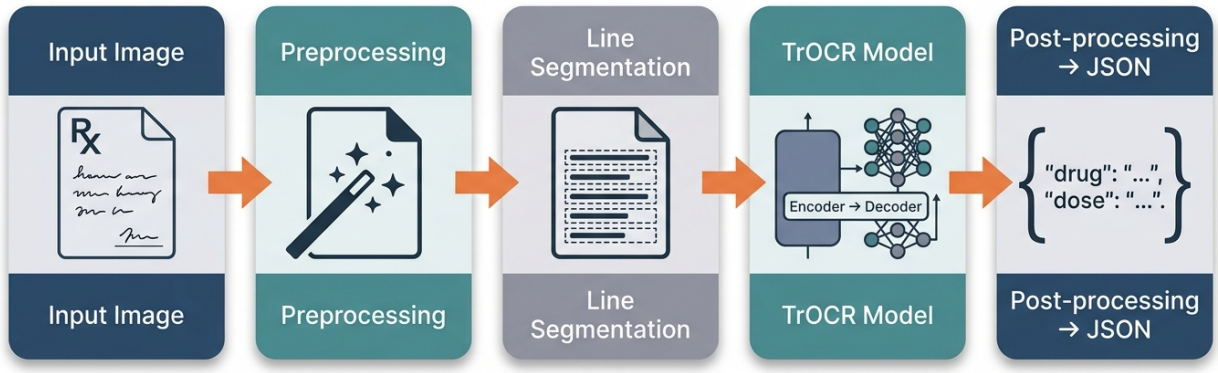

Figure 1: End-to-end medical prescription OCR pipeline.

2. Background: OCR and Vision Encoders

2.1. From Classical OCR to Deep Learning

Classical OCR engines (Tesseract, ABBYY) rely on hand-crafted features and per-character segmentation. They work well on clean printed text but break down on cursive handwriting, where character boundaries are ambiguous. The first deep-learning wave used CNN+RNN+CTC models (e.g. CRNN), which jumped accuracy on scene text but still struggled with long-range context in handwriting.

2.2. The Vision Transformer (ViT)

The Vision Transformer reframed image understanding as a sequence-modeling problem: an image is split into fixed-size patches (e.g. 16×16), each patch is linearly projected into an embedding, positional embeddings are added, and the resulting sequence is fed through a stack of self-attention layers — exactly the architecture used in language Transformers.

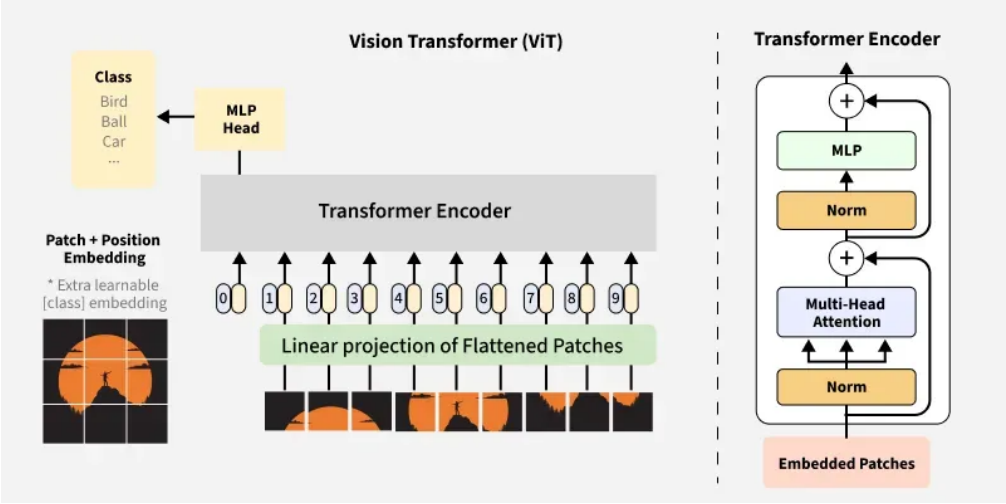

Figure 2: Vision Transformer (ViT): an image is split into patches and processed as a sequence.

2.3. Encoder–Decoder Architectures for Image-to-Text

For OCR, we need not just an image representation but a text generator. The encoder–decoder paradigm fits naturally: the encoder produces a sequence of visual tokens, and an autoregressive decoder generates output tokens one at a time, attending back to the encoder via cross-attention. This is the recipe behind TrOCR, Donut, and most modern document-AI models.

3. TrOCR: Model Architecture

3.1. Overview of TrOCR

TrOCR (*Transformer-based Optical Character Recognition*, Li et al. 2021) is a fully Transformer-based encoder–decoder model. It pairs a pretrained image Transformer encoder (BEiT, ViT, or DeiT) with a pretrained text Transformer decoder (RoBERTa or similar). End-to-end fine-tuning then aligns the two for the OCR task.

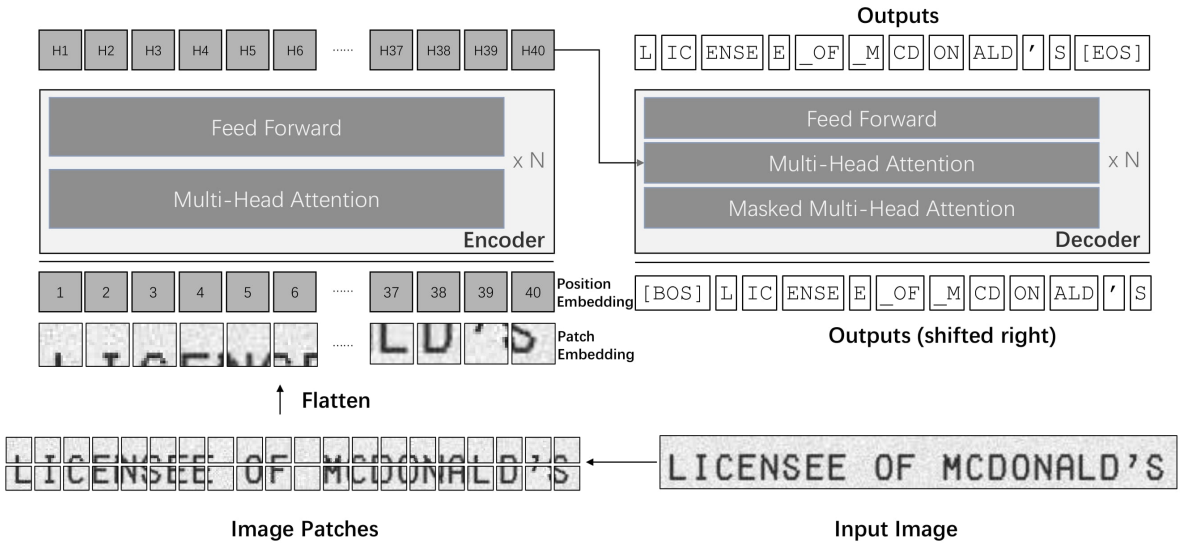

Figure 3: TrOCR: a pretrained image Transformer encoder feeds a pretrained text Transformer decoder.

3.2. The Image Encoder

For `trocr-base-handwritten` the encoder is a BEiT-base model: 12 layers, hidden size 768, 384×384 input resized into 16×16 patches (576 patch tokens + 1 [CLS]). The encoder turns the image into a sequence of visual context vectors.

3.3. The Text Decoder

The decoder is a RoBERTa-style Transformer. At each step it consumes previously generated tokens (teacher-forced during training, autoregressive at inference) and attends to the encoder output through cross-attention to produce the next token.

3.4. Cross-Attention and Generation

Cross-attention is the bridge: decoder query vectors attend to encoder key/value vectors, letting each generated token "look at" the relevant region of the image. At inference we use beam search (beam size 2 in our config) to balance accuracy and latency.

3.5. Why TrOCR for Handwritten Prescriptions

Three reasons:

- It comes with a *handwritten* variant pretrained on IAM, providing a strong starting point for cursive text.
- Its decoder has a real language model prior — helpful for resolving ambiguous strokes using context.
- It plays nicely with HuggingFace tooling, making fine-tuning, checkpointing, and serving straightforward.

4. Dataset

4.1. Source

We use `chinmays18/medical-prescription-dataset` from the HuggingFace Hub: cropped line-level images of handwritten prescription text paired with ground-truth transcriptions.

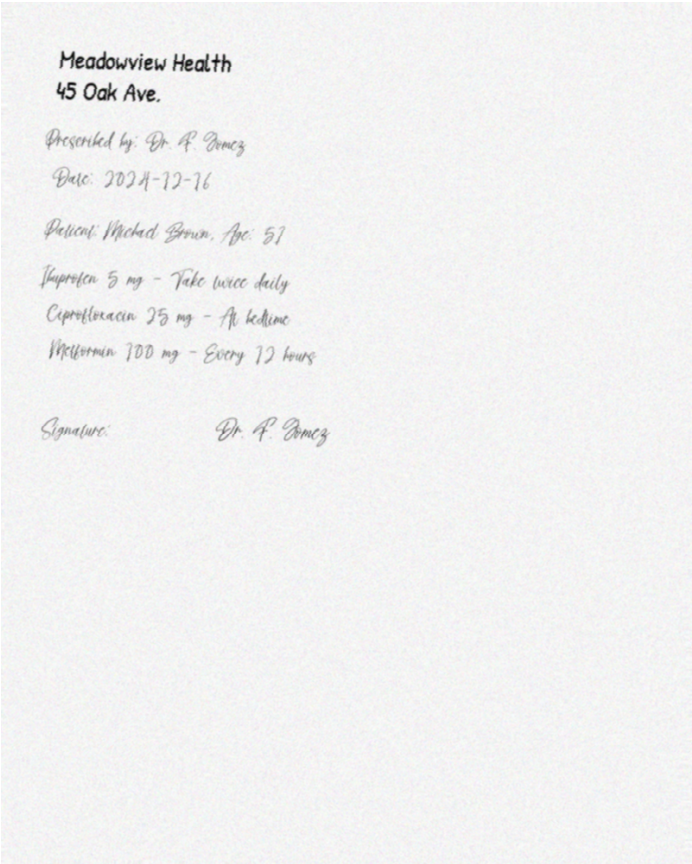

Figure 4: Examples from the prescription dataset: cropped lines with ground-truth labels.

The dataset annotations are structured to facilitate extraction of key clinical entities. Below is a sample of the ground-truth format used for training:

```
{
    "ground_truth": "<s_ocr> doctor_name: Dr. F. Gomez clinic_name: Meadowview
        Health clinic_address: 45 Oak Ave. patient_name: Michael Brown patient_age:
         51 date: 2024-12-16 medications: - Ibuprofen 5 mg - Take twice daily -
        Ciprofloxacin 25 mg - At bedtime - Metformin 100 mg - Every 12 hours
        signature: Dr. F. Gomez </s>"
}
```

4.2. Dataset Statistics

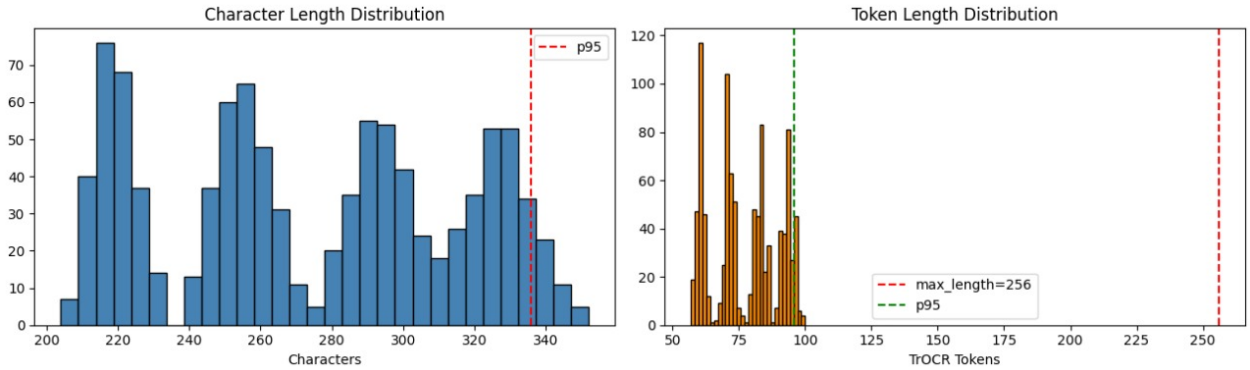

Figure 5: Distribution of character lengths (left) and token lengths (right) across the prescription dataset. The 95th percentile is highlighted in both plots to inform sequence length limits.

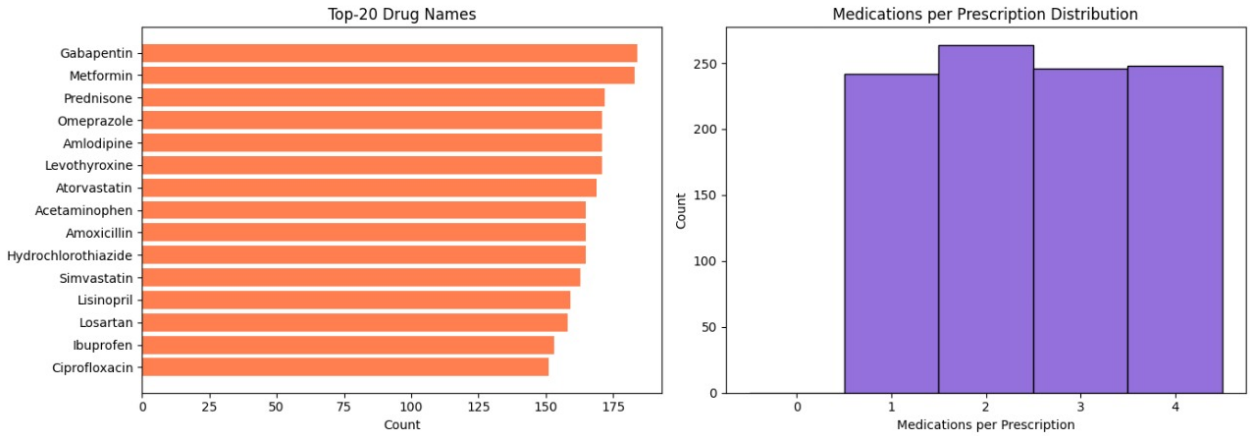

Figure 6: Frequency of the top-20 most commonly prescribed medications (left) and the distribution of the number of medications per prescription (right).

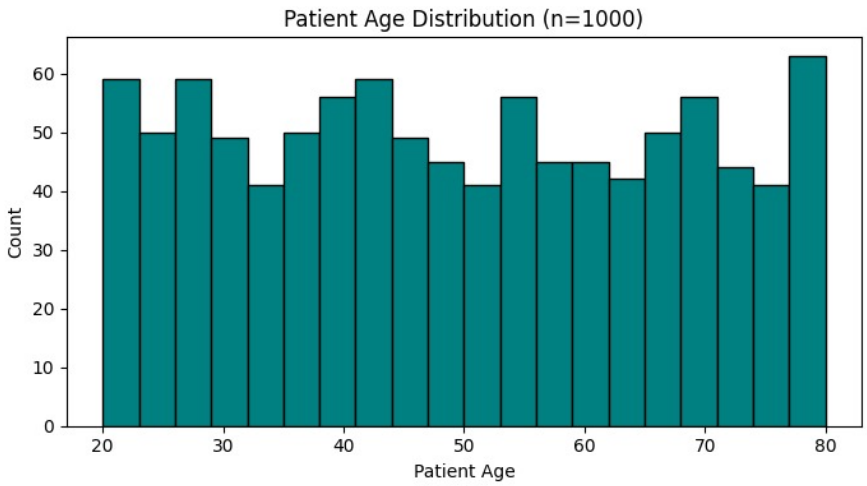

Figure 7: Age distribution of patients represented in the dataset ($n = 1000$).

4.3. Train / Validation / Test Split

The split is controlled by `configs/config.yaml`: 80% train, 10% validation, 10% test. Splits are seeded for reproducibility.

5. Preprocessing and Line Segmentation

5.1. Image Cleanup

Before TrOCR sees a crop, the raw photograph passes through a cleanup stage in `src/preprocess.py`: grayscale conversion, denoising, adaptive binarization, and deskewing.

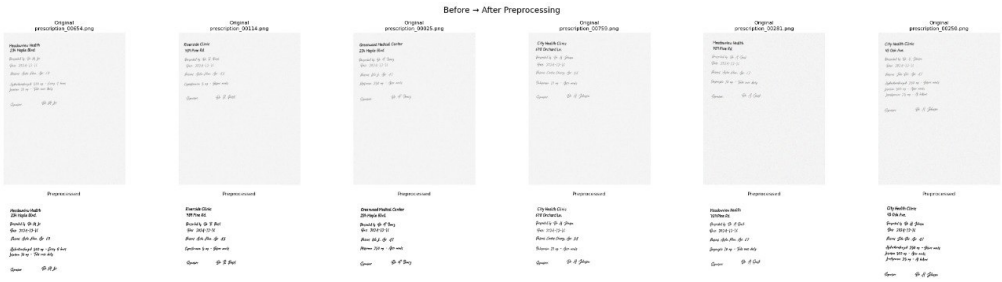

Figure 8: Effect of the preprocessing stage on a noisy phone-captured prescription.

5.2. Line / Region Detection

`src/detection/detector.py` segments the cleaned image into individual text lines using horizontal projection profiles followed by contour-based refinement. Each detected line is cropped and passed independently to TrOCR.

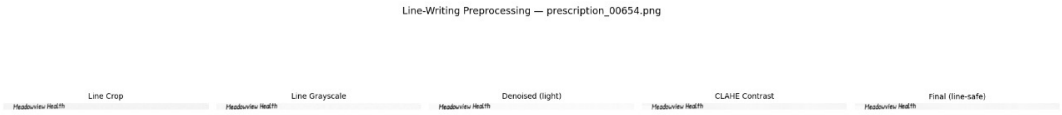

Figure 9: Detected line regions overlaid on the input prescription.

5.3. TrOCR Processor

`TrOCRProcessor` wraps the image processor (resize to 384×384, normalize with ImageNet statistics) and the tokenizer. The same processor is used at training and inference — the artifact is loaded from the fine-tuned checkpoint to guarantee consistency.

6. Training Methodology

6.1. Configuration

All training settings live in `configs/config.yaml` so the same file drives training, evaluation, and inference. Key entries:

configs/config.yaml

```yaml
model:
  name: "microsoft/trocr-base-handwritten"
  max_length: 128

training:
  epochs: 30
  batch_size: 8
  learning_rate: 5e-5
  weight_decay: 0.01
  warmup_steps: 500
  grad_clip: 1.0
  val_interval: 1
  checkpoint_dir: "checkpoints/"

inference:
  beam_size: 2
  device: "auto"
```

6.2. Hardware: Kaggle 2×T4 Setup

Fine-tuning runs on a Kaggle notebook with two T4 GPUs (16 GB each). The notebook automates dataset download, training, evaluation, and pushing the final checkpoint to the HuggingFace Hub.

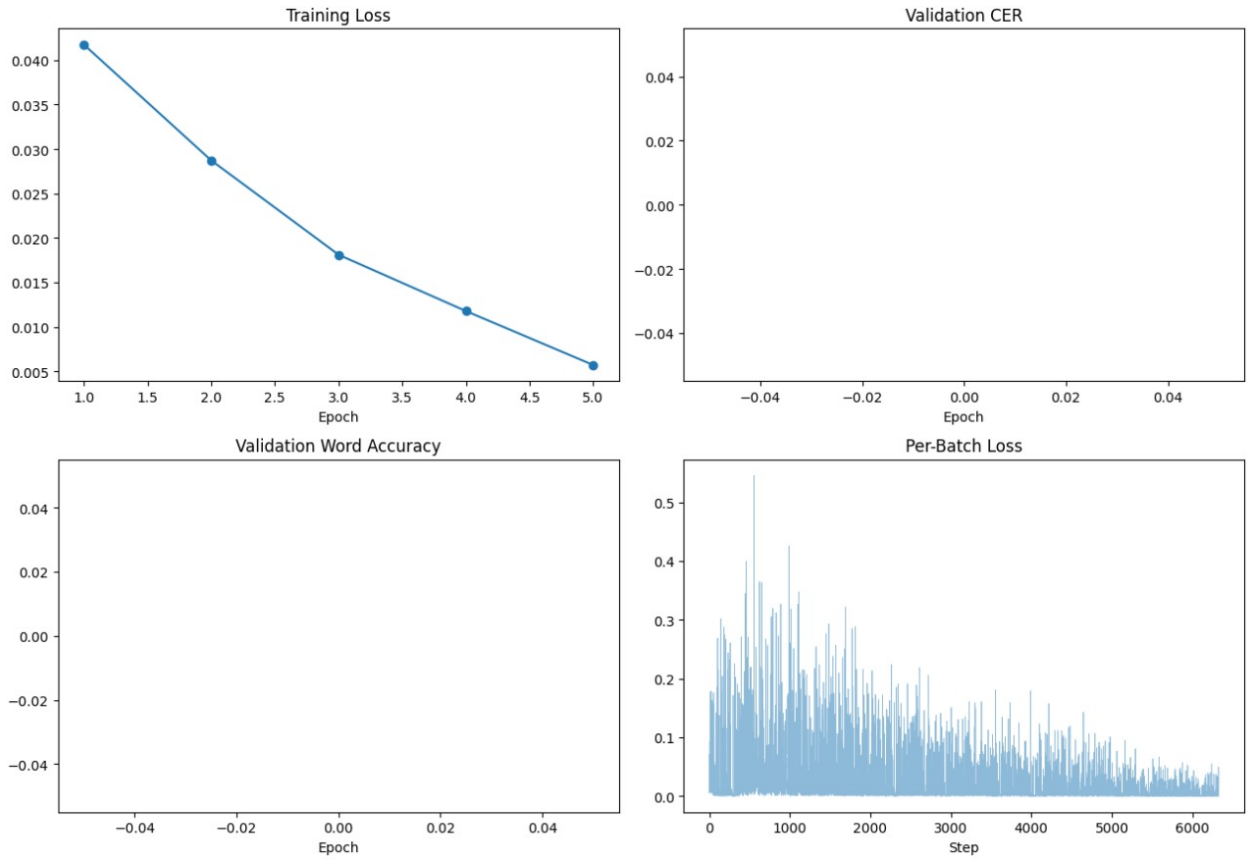

Figure 10: Training metrics tracked over 5 epochs, including per-epoch training loss (top left) and granular per-batch training loss (bottom right), demonstrating stable convergence.

6.3. Loss, Optimizer, and Generation

Standard cross-entropy is computed over decoder logits with teacher forcing during training. The optimizer is AdamW with linear warmup followed by linear decay. Gradient clipping at 1.0 stabilizes training. At inference, generation uses beam search with `num_beams=2` and `max_length=128`.

6.4. Checkpointing and HuggingFace Hub Persistence

After each validation pass, the best checkpoint (by validation CER) is saved to `checkpoints/`. Once training completes, the best checkpoint is pushed to the Hub at khedim/Medical-Prescription-OCR, so deployments only need a single `from_pretrained` call.

7. Inference Pipeline

7.1. End-to-End Flow

A single call to `OCRPipeline.predict(pil_image)` runs:

1. Preprocessing (`src/preprocess.py`).
2. Line detection (`src/detection/detector.py`).
3. Per-line TrOCR generation (`src/model.py:TrOCREngine.generate`).
4. Post-processing into structured fields (`src/postprocess.py`).

OCRPipeline (simplified)

```python
class OCRPipeline:
    @classmethod
    def from_config(cls, path: str) -> "OCRPipeline":
        return cls(_load_config(path))

    def __init__(self, cfg):
        self.preprocessor = Preprocessor(cfg)
        self.detector = LineDetector(cfg)
        self.engine = TrOCREngine(
            model_name=cfg["model"]["name"],
            device=cfg["inference"]["device"],
            max_length=cfg["model"]["max_length"],
            beam_size=cfg["inference"]["beam_size"],
        )

    def predict(self, image):
        cleaned = self.preprocessor(image)
        line_images = self.detector(cleaned)
        preds = [p.strip() for p in self.engine.generate(line_images)]
        return {
            "raw_text": "\n".join(preds),
            "lines": preds,
        }
```

7.2. Post-Processing

Raw decoded lines are normalized with regex rules to flag drug names, dosages (e.g. `500 mg`), and frequencies (`TID`, `BID`). The output JSON keeps both the unstructured text and the structured fields so the frontend can show either.

8. Evaluation and Results

8.1. Metrics

We report three standard OCR metrics:

- **CER (Character Error Rate)**: edit distance normalized by reference length.
- **WER (Word Error Rate)**: same, computed over whitespace-separated tokens.
- **Exact-Match Accuracy**: fraction of lines transcribed perfectly.

8.2. Qualitative Examples

To illustrate the necessity of domain-specific fine-tuning, we compare the baseline model against our fine-tuned model. The baseline TrOCR model fails entirely at structuring the data and frequently hallucinates repetitive numbers or incorrect characters (Figure 11).

Figure 11: Baseline TrOCR (zero-shot) predictions. The model struggles with the domain-specific vocabulary and fails to output the targeted metadata structures, often resulting in severe hallucinations.

Post fine-tuning, the model reliably captures both the raw text and the structured entity prefixes (e.g., `medications:`, `clinic_name:`) required for downstream extraction (Figure 12).

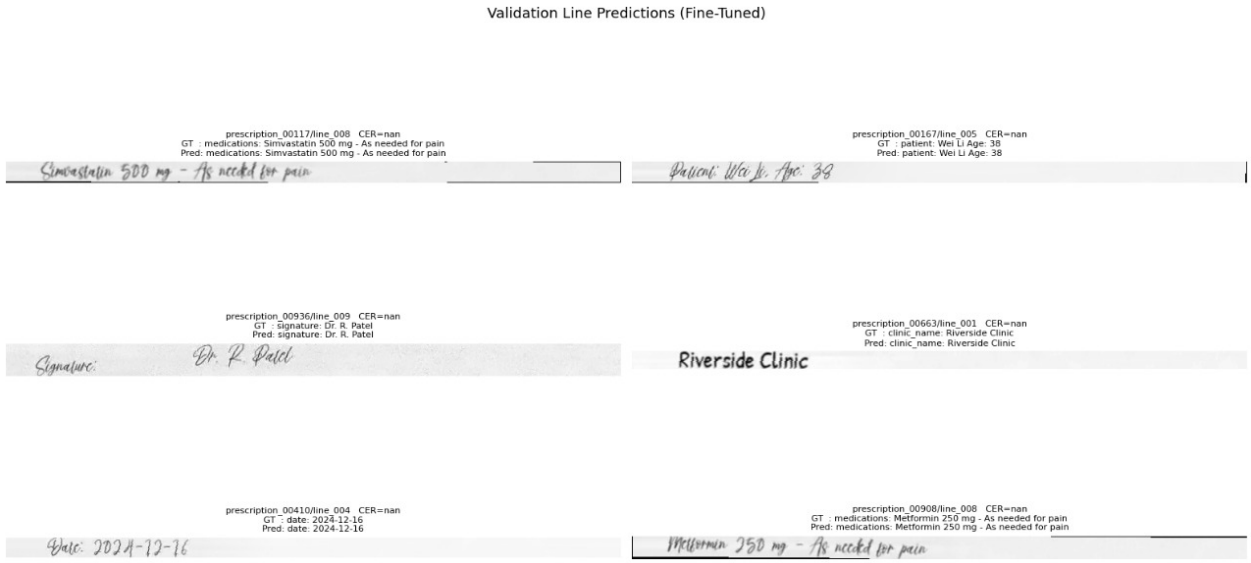

Figure 12: Validation predictions after fine-tuning. The model successfully transcribes cursive handwriting and correctly formats the output into the expected structural schema.

8.3. Failure Analysis

The model still struggles on (a) tightly overlapping cursive strokes, (b) drug names rare in the training set, and (c) crops with strong shadows that survive preprocessing.

9. API and Frontend

9.1. FastAPI Backend

The backend lives in `api/main.py`. It loads the OCR pipeline once at startup and exposes two endpoints:

- `GET /health` — liveness probe, returns model-loaded status.
- `POST /predict` — accepts a multipart image upload, returns the OCR JSON.

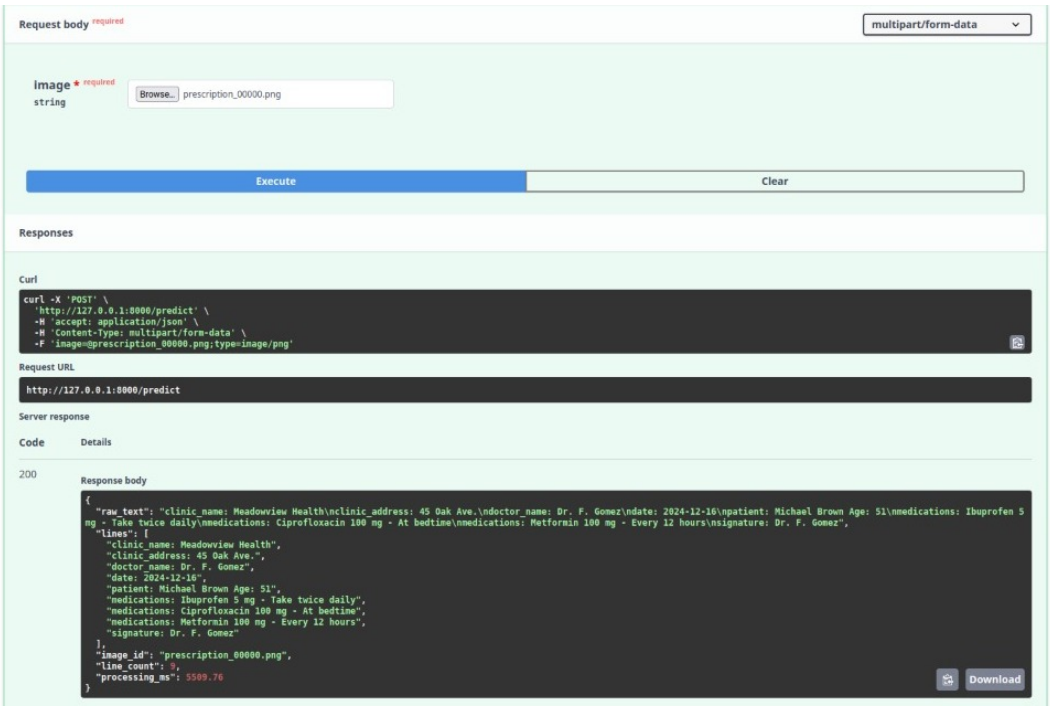

Figure 13: FastAPI auto-generated Swagger UI exposing `/health` and `/predict`.

9.2. React + Vite Frontend

The frontend (`frontend/`) is a Vite-powered React app that uploads an image to `/predict` and renders the predicted lines, total processing time, and raw text.

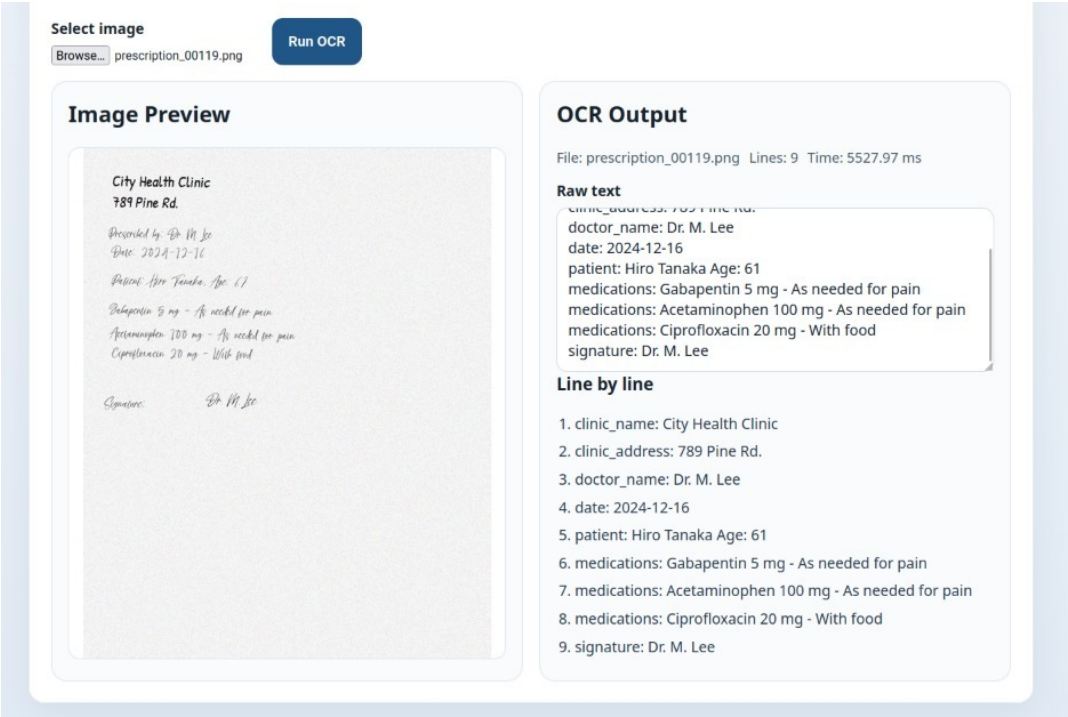

Figure 14: Prediction results view showing extracted lines and processing time.

10. Discussion and Limitations

The fine-tuned model meaningfully outperforms zero-shot TrOCR on prescription-style handwriting, validating the choice of architecture and training recipe. Several limitations remain:

- **Dataset scale.** The public dataset is small relative to the variability of real prescriptions; performance on out-of-distribution handwriting drops.
- **Line segmentation as a separate stage.** Errors there cascade into TrOCR. A layout-aware model (Donut, LayoutLMv3) could remove the segmentation step entirely.
- **No formulary normalization.** Predicted drug names are not matched against a real drug database, so typos in the prediction are not corrected against a known vocabulary.
- **No multilingual support.** The current model is trained on English/Latin-script prescriptions only.

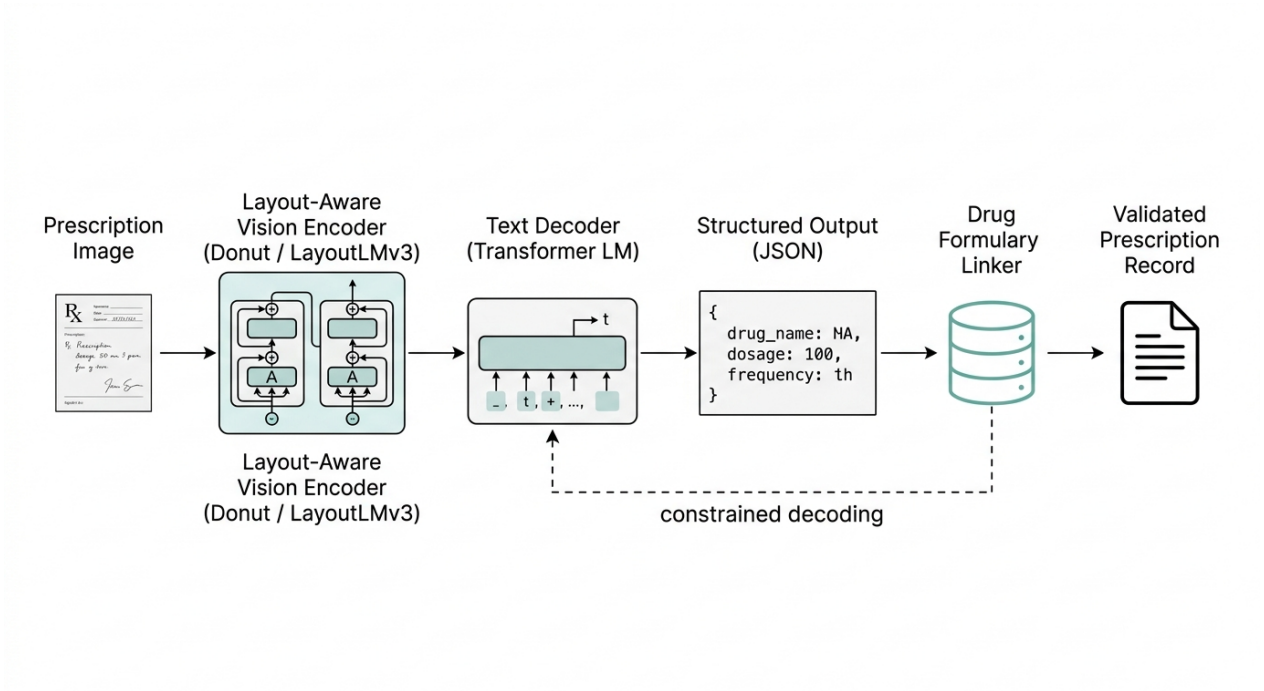

Figure 15: Sketch of a future end-to-end layout-aware OCR pipeline with drug-database normalization.

11. Technologies Used

11.1. PyTorch

PyTorch is the deep-learning framework underlying our entire training and inference stack. We used it for tensor operations, GPU acceleration during training, and model serialization.

11.2. HuggingFace Transformers

The Transformers library provides the `VisionEncoderDecoderModel` and `TrOCRProcessor` classes used throughout the project, plus the Hub integration for checkpoint sharing.

11.3. FastAPI

FastAPI serves the OCR pipeline over HTTP. It auto-generates Swagger documentation, supports asynchronous request handling, and integrates cleanly with Pydantic schemas.

11.4. React + Vite

The frontend is a single-page React application bundled with Vite. It provides image upload, a live preview, and renders the OCR JSON returned by the API.

11.5. Docker

The whole API is shipped as a Docker image so the deployment environment is reproducible across machines.

11.6. Kaggle

Kaggle's free 2×T4 notebook environment hosted the fine-tuning runs. The notebook is automated end-to-end, from dataset download to Hub push.

11.7. GitHub

GitHub hosts the source code and tracks history. Branches and pull requests structured the team workflow.

12. Conclusion

This project shows that a fine-tuned vision encoder–decoder can transcribe handwritten medical prescriptions with practical accuracy, and that the resulting model can be wrapped in a clean API and a usable frontend. TrOCR's combination of a pretrained image encoder and a language-model decoder is well-matched to the noisy, vocabulary-specific nature of prescription text. The remaining gaps — dataset scale, layout awareness, drug-name normalization — point clearly at the next iteration: a larger, curated dataset; a layout-aware end-to-end model such as Donut or LayoutLMv3; and a post-OCR linker that grounds predictions against a real drug formulary.

References

[1] M. Li, T. Lv, J. Chen, et al., *TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models*, arXiv:2109.10282, 2021. `https://arxiv.org/abs/2109.10282`

[2] A. Dosovitskiy et al., *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*, ICLR, 2021. `https://arxiv.org/abs/2010.11929`

[3] H. Bao, L. Dong, S. Piao, F. Wei, *BEiT: BERT Pre-Training of Image Transformers*, ICLR, 2022. `https://arxiv.org/abs/2106.08254`

[4] HuggingFace Transformers documentation. `https://huggingface.co/docs/transformers`

[5] *microsoft/trocr-base-handwritten* model card. `https://huggingface.co/microsoft/trocr-base-handwritten`

[6] *chinmays18/medical-prescription-dataset*. `https://huggingface.co/datasets/chinmays18/medical-prescription-dataset`

[7] S. Ramirez, FastAPI documentation. `https://fastapi.tiangolo.com/`

[8] G. Kim et al., *OCR-free Document Understanding Transformer (Donut)*, ECCV, 2022. `https://arxiv.org/abs/2111.15664`